

1. CREATE DATABASE

```
CREATE DATABASE BankDB;
```

```
USE BankDB;
```

2. CREATE CUSTOMER TABLE

```
CREATE TABLE Customer (  
    Customer_ID INT PRIMARY KEY,  
    Customer_Name VARCHAR(100) NOT NULL,  
    Phone VARCHAR(15),  
    Email VARCHAR(100),  
    City VARCHAR(50)  
);
```

3. CREATE ACCOUNT TABLE

```
CREATE TABLE Account (  
    Account_No INT PRIMARY KEY,  
    Customer_ID INT,  
    Account_Type VARCHAR(20),  
    Balance DECIMAL(12,2) DEFAULT 0,  
    Branch VARCHAR(50),  
  
    FOREIGN KEY (Customer_ID)  
    REFERENCES Customer(Customer_ID)  
);
```

4. CREATE TRANSACTION TABLE

```
CREATE TABLE Bank_Transaction (  
    Transaction_ID INT PRIMARY KEY AUTO_INCREMENT,  
    Account_No INT,  
    Transaction_Type VARCHAR(20),  
    Amount DECIMAL(12,2),  
    Transaction_Date DATETIME DEFAULT CURRENT_TIMESTAMP,  
  
    FOREIGN KEY (Account_No)  
    REFERENCES Account(Account_No)  
);
```

Output				
Action Output				
#	Time	Action	Message	
✓ 1	12:09:27	CREATE DATABASE Bankdb	1 row(s) affected	
✓ 2	12:09:27	USE Bankdb	0 row(s) affected	
✓ 3	12:09:27	CREATE TABLE Customer (Customer_ID INT PRIMARY KEY, Customer_Name VARCHAR(10...	0 row(s) affected	
✓ 4	12:10:11	CREATE TABLE Account (Account_No INT PRIMARY KEY, Customer_ID INT, Account_Typ...	0 row(s) affected	
✓ 5	12:12:08	CREATE TABLE Bank_Transaction(Transaction_ID INT PRIMARY KEY AUTO_INCREMENT...	0 row(s) affected	

5. CREATE LOAN TABLE

```
CREATE TABLE Loan (  
    Loan_ID INT PRIMARY KEY,  
    Customer_ID INT,  
    Loan_Type VARCHAR(30),  
    Loan_Amount DECIMAL(12,2),  
    Interest_Rate DECIMAL(5,2),  
  
    FOREIGN KEY (Customer_ID)  
    REFERENCES Customer(Customer_ID)
```

);

6. INSERT CUSTOMER DATA

INSERT INTO Customer

(Customer_ID, Customer_Name, Phone, Email, City)

VALUES

(101, 'Ravi Kumar', '9876543210', 'ravi@gmail.com', 'Hyderabad'),
(102, 'Priya Sharma', '9876543211', 'priya@gmail.com', 'Vijayawada'),
(103, 'Arjun Reddy', '9876543212', 'arjun@gmail.com', 'Bangalore'),
(104, 'Sneha Rao', '9876543213', 'sneha@gmail.com', 'Chennai'),
(105, 'Kiran Kumar', '9876543214', 'kiran@gmail.com', 'Hyderabad');

Output			
Action Output			
#	Time	Action	Message
✓ 1	12:09:27	CREATE DATABASE Bankdb	1 row(s) affected
✓ 2	12:09:27	USE Bankdb	0 row(s) affected
✓ 3	12:09:27	CREATE TABLE Customer (Customer_ID INT PRIMARY KEY, Customer_Name VARCHAR(10...	0 row(s) affected
✓ 4	12:10:11	CREATE TABLE Account (Account_No INT PRIMARY KEY, Customer_ID INT, Account_Typ...	0 row(s) affected
✓ 5	12:12:08	CREATE TABLE Bank_Transaction(Transaction_ID INT PRIMARY KEY AUTO_INCREMENT...	0 row(s) affected
✓ 6	12:36:05	INSERT INTO Customer (Customer_ID, Customer_Name, Phone, Email, City) VALUES (101, 'R...	5 row(s) affected Records: 5 Duplicates: 0 Warnings: 0

7. INSERT ACCOUNT DATA

INSERT INTO Account

(Account_No, Customer_ID, Account_Type, Balance, Branch)

VALUES

(10001, 101, 'Savings', 50000, 'Hyderabad'),
(10002, 102, 'Savings', 75000, 'Vijayawada'),
(10003, 103, 'Current', 120000, 'Bangalore'),
(10004, 104, 'Savings', 45000, 'Chennai'),
(10005, 105, 'Current', 90000, 'Hyderabad');

```

44 • INSERT INTO Account
45 (Account_No, Customer_ID, Account_Type, Balance, Branch)
46 VALUES (10001, 101, 'Savings', 50000, 'Hyderabad'),
47 (10002, 102, 'Savings', 75000, 'Vijayawada'),
48 (10003, 103, 'Current', 120000, 'Bangalore'),
49 (10004, 104, 'Savings', 45000, 'Chennai'),
50 (10005, 105, 'Current', 90000, 'Hyderabad');
51
52

```

Output					Context Help	Snippets
Action Output						
#	Time	Action	Message			
✓ 2	12:09:27	USE Bankdb	0 row(s) affected			
✓ 3	12:09:27	CREATE TABLE Customer (Customer_ID INT PRIMARY KEY, Customer_Name VARCHAR(1...	0 row(s) affected			
✓ 4	12:10:11	CREATE TABLE Account (Account_No INT PRIMARY KEY, Customer_ID INT, Account_Ty...	0 row(s) affected			
✓ 5	12:12:08	CREATE TABLE Bank_Transaction(Transaction_ID INT PRIMARY KEY AUTO_INCREMEN...	0 row(s) affected			
✓ 6	12:36:05	INSERT INTO Customer (Customer_ID, Customer_Name, Phone, Email, City) VALUES (101, '...	5 row(s) affected Records: 5 Duplicates: 0 Warnings: 0			
✓ 7	12:39:14	INSERT INTO Account (Account_No, Customer_ID, Account_Type, Balance, Branch) VALU...	5 row(s) affected Records: 5 Duplicates: 0 Warnings: 0			

8. INSERT TRANSACTION DATA

INSERT INTO Bank_Transaction

(Account_No, Transaction_Type, Amount)

VALUES

(10001, 'DEPOSIT', 10000),

(10002, 'DEPOSIT', 15000),

(10003, 'WITHDRAW', 20000),

(10004, 'DEPOSIT', 5000),

(10005, 'WITHDRAW', 10000);

✓ 7	12:39:14	INSERT INTO Account (Account_No, Customer_ID, Account_Type, Balance, Branch) VALU...	5 row(s) affected Records: 5 Duplicates: 0 Warnings: 0
✓ 8	12:40:08	INSERT INTO Bank_Transaction (Account_No, Transaction_Type, Amount) VALUES (10001...	5 row(s) affected Records: 5 Duplicates: 0 Warnings: 0

9. INSERT LOAN DATA

INSERT INTO Loan

(Loan_ID, Customer_ID, Loan_Type, Loan_Amount, Interest_Rate)

VALUES

(501, 101, 'Home Loan', 5000000, 7.5),

(502, 102, 'Education Loan', 1000000, 6.5),

(503, 103, 'Car Loan', 800000, 8.2),

(504, 104, 'Personal Loan', 500000, 10.5);

10. DISPLAY TABLE DATA

SELECT * FROM Customer;

Customer_ID	Customer_Name	Phone	Email	City
101	Ravi Kumar	9876543210	ravi@gmail.com	Hyderabad
102	Priya Sharma	9876543211	priya@gmail.com	Vijayawada
103	Arjun Reddy	9876543212	arjun@gmail.com	Bangalore
104	Sneha Rao	9876543213	sneha@gmail.com	Chennai
105	Kiran Kumar	9876543214	kiran@gmail.com	Hyderabad
NULL	NULL	NULL	NULL	NULL

SELECT * FROM Account;

Account_No	Customer_ID	Account_Type	Balance	Branch
10001	101	Savings	50000.00	Hyderabad
10002	102	Savings	75000.00	Vijayawada
10003	103	Current	120000.00	Bangalore
10004	104	Savings	45000.00	Chennai
10005	105	Current	90000.00	Hyderabad
NULL	NULL	NULL	NULL	NULL

SELECT * FROM Bank_Transaction;

Transaction_ID	Account_No	Transaction_Type	Amount	Transaction_Date
1	10001	DEPOSIT	10000.00	2026-08-14 12:40:08
2	10002	DEPOSIT	15000.00	2026-08-14 12:40:08
3	10003	WITHDRAW	20000.00	2026-08-14 12:40:08
4	10004	DEPOSIT	5000.00	2026-08-14 12:40:08
5	10005	WITHDRAW	10000.00	2026-08-14 12:40:08
NULL	NULL	NULL	NULL	NULL

SELECT * FROM Loan;

Result Grid					
Filter Rows:					
Edit:    Export/Imp					
	Loan_ID	Customer_ID	Loan_Type	Loan_Amount	Interest_Rate
▶	501	101	Home Loan	5000000.00	7.50
	502	102	Education Loan	1000000.00	6.50
	503	103	Car Loan	800000.00	8.20
	504	104	Personal Loan	500000.00	10.50
✱	NULL	NULL	NULL	NULL	NULL

11. PROCEDURE TO DISPLAY ALL CUSTOMERS

```
CREATE PROCEDURE GetAllCustomers()
```

```
BEGIN
```

```
    SELECT * FROM Customer;
```

```
DELIMITER ;
```

```
EXECUTION:
```

```
CALL GetAllCustomers();
```

Result Grid					
Filter Rows:					
Export:  Wrap Cell Content: 					
	Customer_ID	Customer_Name	Phone	Email	City
▶	101	Ravi Kumar	9876543210	ravi@gmail.com	Hyderabad
	102	Priya Sharma	9876543211	priya@gmail.com	Vijayawada
	103	Arjun Reddy	9876543212	arjun@gmail.com	Bangalore
	104	Sneha Rao	9876543213	sneha@gmail.com	Chennai
	105	Kiran Kumar	9876543214	kiran@gmail.com	Hyderabad

12. PROCEDURE TO DISPLAY ACCOUNT DETAILS

```
FOR A GIVEN ACCOUNT NUMBER
```

```
CREATE PROCEDURE GetAccountDetails(
```

```
    IN p_Account_No INT
```

```
)
```

```
BEGIN
```

```
    SELECT *
```

FROM Account

WHERE Account_No = p_Account_No;

DELIMITER ;

EXECUTION:

CALL GetAccountDetails(10001);

Result Grid					
		Filter Rows:		Export:	Wrap Cell Content:
	Account_No	Customer_ID	Account_Type	Balance	Branch
▶	10001	101	Savings	50000.00	Hyderabad

13. PROCEDURE TO FIND CUSTOMER ACCOUNTS

DELIMITER //

CREATE PROCEDURE GetCustomerAccounts(

IN p_Customer_ID INT

)

BEGIN

SELECT

C.Customer_ID,

C.Customer_Name,

A.Account_No,

A.Account_Type,

A.Balance,

A.Branch

FROM Customer C

JOIN Account A

ON C.Customer_ID = A.Customer_ID

WHERE C.Customer_ID = p_Customer_ID;

END //

DELIMITER ;

EXECUTION:

CALL GetCustomerAccounts(101);

Result Grid						
Filter Rows:		Export:  Wrap Cell Content: 				
	Customer_ID	Customer_Name	Account_No	Account_Type	Balance	Branch
▶	101	Ravi Kumar	10001	Savings	50000.00	Hyderabad

14. PROCEDURE TO DEPOSIT MONEY

DELIMITER //

CREATE PROCEDURE DepositMoney(

IN p_Account_No INT,

IN p_Amount DECIMAL(12,2)

)

BEGIN

UPDATE Account

SET Balance = Balance + p_Amount

WHERE Account_No = p_Account_No;

END //

DELIMITER ;

EXECUTION:

CALL DepositMoney(10001, 5000);

✓	38	13:51:01	CREATE PROCEDURE DepositMoney(IN p_Account_No INT, IN p_Amount DECIMAL(12,2) ...	0 row(s) affected
✓	39	13:51:01	CALL DepositMoney(10001, 5000)	1 row(s) affected

15. PROCEDURE TO WITHDRAW MONEY

DELIMITER //

CREATE PROCEDURE WithdrawMoney(

IN p_Account_No INT,

IN p_Amount DECIMAL(12,2)

)

BEGIN

UPDATE Account

SET Balance = Balance - p_Amount

WHERE Account_No = p_Account_No;

END //

DELIMITER ;

EXECUTION:

CALL WithdrawMoney(10001, 3000);

✓	40	13:52:37	CREATE PROCEDURE WithdrawMoney(IN p_Account_No INT, IN p_Amount DECIM...	0 row(s) affected
✓	41	13:52:37	CALL WithdrawMoney(10001, 3000)	1 row(s) affected

TRIGGERS

16. TRIGGER – PREVENT NEGATIVE BALANCE

DELIMITER //

CREATE TRIGGER CheckBalance

BEFORE UPDATE ON Account

FOR EACH ROW

BEGIN

```

IF NEW.Balance < 0 THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT =
        'Transaction failed: Insufficient balance';
END IF;
END //
DELIMITER ;
TEST:

```

```

UPDATE Account
SET Balance = Balance - 60000
WHERE Account_No = 10001;

```

Expected Result:

Transaction failed: Insufficient balance

 43 13:57:53 UPDATE Account SET Balance = Balance - 60000 WHERE Account_No = 10001	Error Code: 1644. Transaction failed: Insufficient balance
---	--

17. TRIGGER – PREVENT NEGATIVE DEPOSIT

```
DELIMITER //
```

```

CREATE TRIGGER CheckTransactionAmount
BEFORE INSERT ON Bank_Transaction
FOR EACH ROW
BEGIN
    IF NEW.Amount <= 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT =

```

```

        'Transaction amount must be greater than zero';
    END IF;
END //
DELIMITER ;
TEST:
INSERT INTO Bank_Transaction
(Account_No, Transaction_Type, Amount)
VALUES
(10001, 'DEPOSIT', -5000);

```

✖ 45 13:59:52 INSERT INTO Bank_Transaction (Account_No, Transaction_Type, Amount) VALUES (1000... Error Code: 1644. Transaction amount must be greater than zero

18. CREATE TRANSACTION AUDIT TABLE

```

CREATE TABLE Transaction_Audit (
    Audit_ID INT PRIMARY KEY AUTO_INCREMENT,
    Transaction_ID INT,
    Account_No INT,
    Transaction_Type VARCHAR(20),
    Amount DECIMAL(12,2),
    Audit_Date DATETIME DEFAULT CURRENT_TIMESTAMP
);

```

✔ 46 14:01:12 CREATE TABLE Transaction_Audit (Audit_ID INT PRIMARY KEY AUTO_INCREMENT, ... 0 row(s) affected

19. TRIGGER – TRANSACTION AUDIT

```

DELIMITER //
CREATE TRIGGER TransactionAudit
AFTER INSERT ON Bank_Transaction
FOR EACH ROW
BEGIN
    INSERT INTO Transaction_Audit
    (

```

```

Transaction_ID,
Account_No,
Transaction_Type,
Amount
)
VALUES
(
NEW.Transaction_ID,
NEW.Account_No,
NEW.Transaction_Type,
NEW.Amount
);
END //
DELIMITER ;
TEST:

INSERT INTO Bank_Transaction
(Account_No, Transaction_Type, Amount)
VALUES
(10001, 'DEPOSIT', 2500);

```

```

CHECK:
SELECT * FROM Transaction_Audit;

```

✓	49	14:02:31	SELECT * FROM Transaction_Audit LIMIT 0, 1000	1 row(s) returned
---	----	----------	---	-------------------

20. TRIGGER – AUTOMATICALLY UPDATE ACCOUNT BALANCE

```

DELIMITER //
CREATE TRIGGER UpdateBalanceAfterTransaction
AFTER INSERT ON Bank_Transaction

```

FOR EACH ROW

BEGIN

IF NEW.Transaction_Type = 'DEPOSIT' THEN

UPDATE Account

SET Balance = Balance + NEW.Amount

WHERE Account_No = NEW.Account_No;

ELSEIF NEW.Transaction_Type = 'WITHDRAW' THEN

UPDATE Account

SET Balance = Balance - NEW.Amount

WHERE Account_No = NEW.Account_No;

END IF;

END //

DELIMITER ;

TEST:

INSERT INTO Bank_Transaction

(Account_No, Transaction_Type, Amount)

VALUES

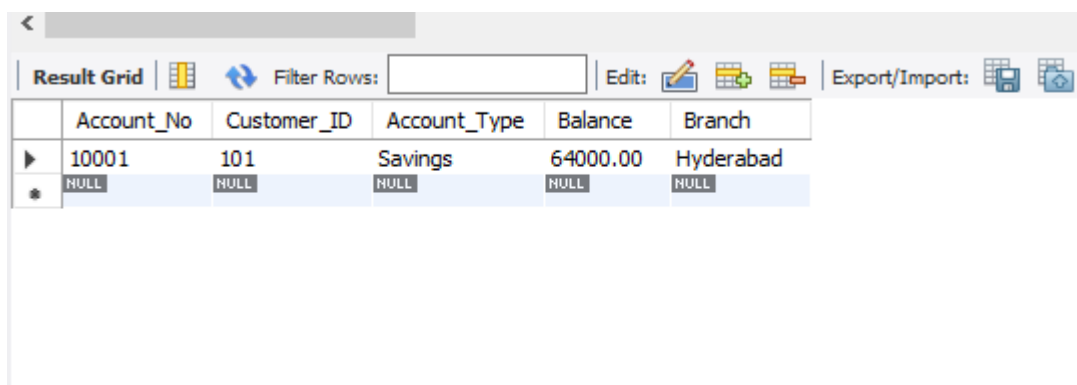
(10001, 'DEPOSIT', 5000);

CHECK:

SELECT *

FROM Account

WHERE Account_No = 10001;



The screenshot shows a database application interface. At the top, there is a toolbar with icons for 'Result Grid', 'Filter Rows', 'Edit', and 'Export/Import'. Below the toolbar is a table with the following columns: Account_No, Customer_ID, Account_Type, Balance, and Branch. The table contains two rows: one with values 10001, 101, Savings, 64000.00, and Hyderabad; and another row with NULL values. A cursor is positioned on the first row.

Account_No	Customer_ID	Account_Type	Balance	Branch
10001	101	Savings	64000.00	Hyderabad
NULL	NULL	NULL	NULL	NULL

21. TRIGGER – PREVENT WITHDRAWAL ABOVE BALANCE

```

DELIMITER //

CREATE TRIGGER PreventInsufficientWithdrawal

BEFORE INSERT ON Bank_Transaction

FOR EACH ROW

BEGIN

    DECLARE CurrentBalance DECIMAL(12,2);

    SELECT Balance

    INTO CurrentBalance

    FROM Account

    WHERE Account_No = NEW.Account_No;

    IF NEW.Transaction_Type = 'WITHDRAW'

        AND NEW.Amount > CurrentBalance THEN

        SIGNAL SQLSTATE '45000'

        SET MESSAGE_TEXT =

            'Withdrawal failed: Insufficient balance';

    END IF;

END //

DELIMITER ;

TEST:

INSERT INTO Bank_Transaction

(Account_No, Transaction_Type, Amount)

VALUES

(10001, 'WITHDRAW', 1000000);

```

✖ 44 19:11:56 INSERT INTO Bank_Transaction (Account_No, Transaction_Type, Amount) VALUES (1000... Error Code: 1644. Withdrawal failed: Insufficient balance

22. PROCEDURE – ACCOUNT TRANSFER

```

DELIMITER //

CREATE PROCEDURE TransferMoney(

    IN SenderAccount INT,

```

```
    IN ReceiverAccount INT,  
    IN TransferAmount DECIMAL(12,2)  
)  
BEGIN  
  
    DECLARE SenderBalance DECIMAL(12,2);  
  
    SELECT Balance  
    INTO SenderBalance  
    FROM Account  
    WHERE Account_No = SenderAccount;  
  
    IF SenderBalance < TransferAmount THEN  
  
        SIGNAL SQLSTATE '45000'  
        SET MESSAGE_TEXT =  
        'Transfer failed: Insufficient balance';  
  
    ELSE  
  
        UPDATE Account  
        SET Balance = Balance - TransferAmount  
        WHERE Account_No = SenderAccount;  
  
        UPDATE Account  
        SET Balance = Balance + TransferAmount  
        WHERE Account_No = ReceiverAccount;  
  
    END IF;  
  
END //
```

DELIMITER ;

EXECUTION:

CALL TransferMoney(10001, 10002, 5000);

CHECK:

SELECT *

FROM Account

WHERE Account_No IN (10001,10002);

Result Grid			Filter Rows:	Edit:   	E
	Account_No	Customer_ID	Account_Type	Balance	Branch
	10001	101	Savings	59000.00	Hyderabad
	10002	102	Savings	80000.00	Vijayawada
*	NULL	NULL	NULL	NULL	NULL

23. PROCEDURE – CUSTOMER LOAN DETAILS

DELIMITER //

CREATE PROCEDURE GetCustomerLoans(

IN p_Customer_ID INT

)

BEGIN

SELECT


```

        C.Customer_Name,
        L.Loan_ID,
        L.Loan_Type,
        L.Loan_Amount,
        L.Interest_Rate
    FROM Customer C
    JOIN Loan L
    ON C.Customer_ID = L.Customer_ID
    WHERE C.Customer_ID = p_Customer_ID;
END //
DELIMITER ;
EXECUTION:
CALL GetCustomerLoans(101);

```

✓	48	19:14:06	CREATE PROCEDURE GetCustomerLoans(IN p_Customer_ID INT) BEGIN	SELECT ...	0 row(s) affected
✓	49	19:14:06	CALL GetCustomerLoans(101)		1 row(s) returned

24. PROCEDURE – HIGH BALANCE ACCOUNTS

```
DELIMITER //
```

```

CREATE PROCEDURE HighBalanceAccounts(
    IN MinimumBalance DECIMAL(12,2)
)

```

```
BEGIN
```

```

    SELECT *
    FROM Account
    WHERE Balance >= MinimumBalance
    ORDER BY Balance DESC;

```

```
END //
```

DELIMITER ;

EXECUTION:

CALL HighBalanceAccounts(50000);

✓ 51 19:15:19 CALL HighBalanceAccounts(50000) 4 row(s) returned

25. PROCEDURE WITH IN, OUT PARAMETERS

DELIMITER //

CREATE PROCEDURE GetBalance(

IN p_Account_No INT,

OUT p_Balance DECIMAL(12,2)

)

BEGIN

SELECT Balance

INTO p_Balance

FROM Account

WHERE Account_No = p_Account_No;

END //

DELIMITER ;

EXECUTION:

CALL GetBalance(10001, @CurrentBalance);

SELECT @CurrentBalance;

✓	54	19:15:55	SELECT @CurrentBalance LIMIT 0, 1000	1 row(s) returned
---	----	----------	--------------------------------------	-------------------